

■ Lab 15 — Advanced Terraform & CI/CD avec GitHub Actions

■ Objectifs du Lab

À la fin de ce lab, vous aurez :

- Compris et utilisé le **dependency lock file** (`terraform.lock.hcl`)
- Renommé une ressource avec le bloc `moved` sans la détruire/recréer
- Validé l'infrastructure avec le bloc `check` (assertions post-apply)
- Géré les **secrets AWS** de manière sécurisée via les variables CI/CD
- Utilisé `terraform plan -out=planfile` + `terraform apply planfile`
- Mis en place un pipeline **CI/CD complet** avec GitHub Actions

■ Étape 1 — Créer les fichiers

```
cd labs/lab15-advanced
```

``backend.tf``

```
terraform {  
  backend "s3" {  
    bucket      = "tf-training-<votre-prenom>-982908300187"  
    key         = "terraform.tfstate"  
    region     = "eu-west-1"  
    use_lockfile = true  
    encrypt     = true  
  }  
}
```

■ ■ Remplacez `` par votre prénom. Ex : `tf-training-alice-982908300187`

``versions.tf``

```
terraform {
  required_version = ">= 1.15.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}
```

`provider.tf`

```
provider "aws" {
  region = var.region
}
```

`variables.tf`

```
variable "username" {
  description = "Votre prenom"
  type        = string
}

variable "region" {
  description = "Region AWS"
  type        = string
  default     = "eu-west-1"
}

variable "environment" {
  description = "Environnement de déploiement"
  type        = string
  default     = "dev"

  validation {
    condition     = contains(["dev", "prod"], var.environment)
    error_message = "L'environnement doit être dev ou prod."
  }
}
```

`locals.tf`

```
locals {
  prefix = "lab15-${var.username}-${var.environment}"

  common_tags = {
    Lab          = "lab15"
    Username     = var.username
    Environment  = var.environment
    ManagedBy    = "Terraform"
  }
}
```

`main.tf`

```
data "aws_ami" "amazon_linux" {
  most_recent = true
  owners      = ["amazon"]
  filter {
    name     = "name"
    values   = ["al2023-ami-*-x86_64"]
  }
}

resource "aws_security_group" "lab15_sg" {
  name        = "${local.prefix}-sg"
  description = "Security group Lab 15 - ${var.username}"

  ingress {
    from_port = 80
    to_port   = 80
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
    description = "HTTP"
  }

  egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = merge(local.common_tags, {
    Name = "${local.prefix}-sg"
  })
}
```

```
    })
  }

resource "aws_instance" "lab15_instance" {
  ami              = data.aws_ami.amazon_linux.id
  instance_type    = "t2.micro"
  vpc_security_group_ids = [aws_security_group.lab15_sg.id]
  associate_public_ip_address = true

  tags = merge(local.common_tags, {
    Name = "${local.prefix}-instance"
  })
}
```

`outputs.tf`

```
output "instance_id" {
  description = "ID de l'instance"
  value       = aws_instance.lab15_instance.id
}

output "instance_public_ip" {
  description = "IP publique de l'instance"
  value       = aws_instance.lab15_instance.public_ip
}

output "security_group_id" {
  description = "ID du Security Group"
  value       = aws_security_group.lab15_sg.id
}
```

■ Étape 2 — Dependency Lock File (`terraform.lock.hcl`)

```
terraform init
```

Inspectez le fichier généré :

```
cat .terraform.lock.hcl
```

Résultat attendu :

```
provider "registry.terraform.io/hashicorp/aws" {  
  version      = "5.x.x"  
  constraints = "~> 5.0"  
  hashes = [  
    "h1:xxxx...",  
  ]  
}
```

■ Le fichier `.terraform.lock.hcl` **verrouille les versions exactes** des providers utilisés.

Il doit être **commité dans Git** pour garantir que tous les membres de l'équipe et le CI/CD utilisent exactement les mêmes versions.

Ne jamais l'ajouter au `.gitignore` !

```
# Mettre à jour le lock file si nécessaire  
terraform init -upgrade
```

■ Étape 3 — Plan file : ``terraform plan -out``

La bonne pratique en CI/CD est de **séparer le plan et l'apply** :

```
terraform init  
  
# Étape 1 : Générer le plan et le sauvegarder dans un fichier  
terraform plan \  
  -var="username=<votre-prenom>" \  
  -out=tfplan
```

```
# Inspecter le plan sauvegardé  
terraform show tfplan
```

```
# Étape 2 : Appliquer exactement ce qui a été planifié  
terraform apply tfplan
```

■ Cette approche garantit que ce qui a été **validé** lors du plan est **exactement** ce qui sera appliqué — aucune surprise si l'infrastructure a changé entre les deux.

C'est le pattern standard en CI/CD : ``plan`` dans la PR, ``apply`` après merge.

■■ Le fichier ``tfplan`` contient des informations sensibles (credentials, outputs) — ne jamais le commiter dans Git. Ajoutez ``tfplan`` à votre `.gitignore``.

■ Étape 4 — Bloc `moved` (renommer sans recréer)

Étape 4.1 — Situation initiale

Après l'apply, votre state contient :

```
aws_instance.lab15_instance
aws_security_group.lab15_sg
```

Étape 4.2 — Renommer la ressource dans le code

Dans `main.tf`, renommez `lab15_instance` en `web_server` :

```
resource "aws_instance" "web_server" {    # ← renommé
  ami                = data.aws_ami.amazon_linux.id
  instance_type      = "t2.micro"
  vpc_security_group_ids = [aws_security_group.lab15_sg.id]
  associate_public_ip_address = true

  tags = merge(local.common_tags, {
    Name = "${local.prefix}-instance"
  })
}
```

Mettez aussi à jour `outputs.tf` :

```
output "instance_id" {
  value = aws_instance.web_server.id
}

output "instance_public_ip" {
  value = aws_instance.web_server.public_ip
}
```

Étape 4.3 — Ajouter le bloc `moved`

Créez `moved.tf` :

```
moved {
  from = aws_instance.lab15_instance
  to   = aws_instance.web_server
}
```

Étape 4.4 — Vérifier que Terraform ne recrée pas la ressource

```
terraform plan -var="username=<votre-prenom>"
```

Résultat attendu :

```
# aws_instance.lab15_instance has moved to aws_instance.web_server
Plan: 0 to add, 0 to change, 0 to destroy.
```

■ Sans le bloc `moved`, Terraform aurait **détruit** `lab15_instance` et **recréé** `web_server` — ce qui cause une interruption de service. Le bloc `moved` l'informe que c'est un simple renommage.

```
terraform apply -var="username=<votre-prenom>"
```

■ Étape 5 — Bloc `check` (assertions post-apply)

Le bloc `check` permet de **valider l'état de l'infrastructure** après le déploiement.

Ajoutez dans `main.tf` :

```
check "instance_is_running" {
  data "aws_instance" "verify" {
    instance_id = aws_instance.web_server.id
  }

  assert {
    condition     = data.aws_instance.verify.instance_state == "running"
    error_message = "L'instance ${aws_instance.web_server.id} n'est pas en état running !"
  }
}

check "security_group_exists" {
  data "aws_security_group" "verify" {
    id = aws_security_group.lab15_sg.id
  }

  assert {
    condition     = data.aws_security_group.verify.id != ""
    error_message = "Le Security Group n'existe pas ou n'est pas accessible !"
  }
}
```

```
terraform apply -var="username=<votre-prenom>"
```

■ Contrairement aux validations de variables, le bloc `check` s'exécute **après** le déploiement et vérifie l'état réel des ressources. Un échec affiche un **warning** mais n'arrête pas l'apply.

■ Étape 6 — Gestion des secrets (bonnes pratiques)

■ ****Règle absolue : ne jamais mettre de secrets dans `terraform.tfvars` ou dans le code.****

Ce qu'il ne faut jamais faire

```
# ■ INTERDIT – ne jamais faire ça
variable "aws_access_key" {
  default = "AKIAXXXXXXXXXXXXXXXX"
}
```

La bonne approche : variables d'environnement

```
# ■ Credentials via variables d'environnement
export AWS_ACCESS_KEY_ID="AKIAXXXXXXXXXXXXXXXX"
export AWS_SECRET_ACCESS_KEY="XXXXXXXXXXXXXXXXXXXXXXXXXXXX"
```

En CI/CD : GitHub Actions Secrets

Les secrets sont stockés dans **GitHub** → **Settings** → **Secrets and variables** → **Actions** et injectés comme variables d'environnement dans le pipeline — jamais en clair dans le code.

■ Les secrets GitHub Actions sont ****chiffrés****, non visibles dans les logs, et non accessibles en dehors des workflows.

■ Étape 7 — CI/CD avec GitHub Actions

Étape 7.1 — Configurer les secrets GitHub

Dans votre repo GitHub :

1. **Settings** → **Secrets and variables** → **Actions**
2. Cliquez **New repository secret**
3. Ajoutez les deux secrets :

- `AWS_ACCESS_KEY_ID` → votre Access Key ID
- `AWS_SECRET_ACCESS_KEY` → votre Secret Access Key

Étape 7.2 — Créer le workflow


```
mkdir -p .github/workflows
```

Créez `.github/workflows/terraform.yml` :

```
name: Terraform CI/CD

on:
  push:
    branches: [ main ]
    paths: [ 'labs/lab15-advanced/**' ]
  pull_request:
    branches: [ main ]
    paths: [ 'labs/lab15-advanced/**' ]
  workflow_dispatch:
    inputs:
      action:
        description: 'Action à exécuter'
        required: true
        default: 'plan'
        type: choice
        options: [ plan, apply, destroy ]

env:
  TF_WORKING_DIR: labs/lab15-advanced
  TF_VAR_username: ${ github.actor }

jobs:
  terraform:
    name: Terraform ${ github.event.inputs.action || 'plan' }
    runs-on: ubuntu-latest

    defaults:
      run:
        working-directory: ${ env.TF_WORKING_DIR }
```

```

steps:
  - name: Checkout code
    uses: actions/checkout@v4

  - name: Setup Terraform
    uses: hashicorp/setup-terraform@v3
    with:
      terraform_version: "1.15.5"

  - name: Configure AWS credentials
    uses: aws-actions/configure-aws-credentials@v4
    with:
      aws-access-key-id: ${ secrets.AWS_ACCESS_KEY_ID }
      aws-secret-access-key: ${ secrets.AWS_SECRET_ACCESS_KEY }
      aws-region: eu-west-1

  - name: Terraform Init
    run: terraform init

  - name: Terraform Format Check
    run: terraform fmt -check -recursive

  - name: Terraform Validate
    run: terraform validate

  - name: Terraform Plan
    run: |
      terraform plan \
        -var="environment=dev" \

      -out=tfplan
      if: github.event_name == 'pull_request' || github.event.inputs.action == 'plan' || github.event_

  - name: Upload Plan
    uses: actions/upload-artifact@v4
    with:
      name: tfplan
      path: ${ env.TF_WORKING_DIR }}/tfplan
      retention-days: 1
      if: github.event_name == 'pull_request' || github.event.inputs.action == 'plan' || github.event_

  - name: Terraform Apply
    run: terraform apply tfplan
    if: github.event.inputs.action == 'apply'

  - name: Terraform Destroy
    run: |
      terraform plan -destroy -var="environment=dev" -out=tfplan-destroy
      terraform apply tfplan-destroy
      if: github.event.inputs.action == 'destroy'

```

Étape 7.3 — Commiter et pousser

```
git add .  
git commit -m "feat: lab15 advanced terraform + CI/CD"  
git push origin main
```

Étape 7.4 — Observer le pipeline

1. Allez sur votre repo GitHub → onglet **Actions**
2. Vous voyez le pipeline **Terraform CI/CD** qui s'exécute
3. Cliquez dessus pour voir les logs de chaque étape

■ Étape 8 — Tester le workflow complet

Scénario 1 — Plan automatique sur Push

```
# Modifier un tag dans main.tf  
# Puis commiter et pousser  
git add .  
git commit -m "test: modifier un tag pour déclencher le pipeline"  
git push origin main
```

■ *Le pipeline se déclenche automatiquement et exécute `terraform plan`.*

Scénario 2 — Apply manuel via workflow_dispatch

1. GitHub → **Actions** → **Terraform CI/CD**
2. Cliquez **Run workflow**
3. Sélectionnez **apply** dans le menu déroulant
4. Cliquez **Run workflow**

Scénario 3 — Destroy via workflow_dispatch

1. GitHub → **Actions** → **Terraform CI/CD**
2. Cliquez **Run workflow**
3. Sélectionnez **destroy**
4. Cliquez **Run workflow**

■ Étape 9 — Nettoyage

```
# Via le workflow GitHub Actions (destroy)
# OU localement :
terraform destroy -var="username=<votre-prenom>"
rm -f tfplan tfplan-destroy
```

■ Validation du Lab

#	Critère	Validé
1	`terraform.lock.hcl` présent et commité dans Git	■
2	`terraform plan -out=tfplan` + `terraform apply tfplan` fonctionne	■
3	Bloc `moved` renomme la ressource sans la recréer (`0 to destroy`)	■
4	Bloc `check` valide l'état de l'instance après apply	■
5	Secrets AWS dans GitHub Actions Secrets (jamais dans le code)	■
6	Pipeline GitHub Actions se déclenche sur push	■
7	`terraform fmt -check` passe dans le pipeline	■
8	`terraform validate` passe dans le pipeline	■
9	Apply manuel via `workflow_dispatch` fonctionne	■
10	Destroy via `workflow_dispatch` supprime les ressources	■

■■ Résolution des problèmes courants

Problème	Cause	Solution
`lock file not committed`	`.terraform.lock.hcl` dans `.gitignore`	Retirer du `.gitignore` et commiter
`plan file stale`	State modifié entre plan et apply	Relancer `terraform plan -out=tfplan`
`moved block` recrée la ressource	Mauvais nom dans `from`	Vérifier avec `terraform state list`
`check` échoue en warning	Instance pas encore running	Attendre 30s et relancer `terraform apply`
Secrets visibles dans les logs	`echo` ou `print` d'une variable	Ne jamais afficher les variables sensibles
Pipeline échoue sur `fmt -check`	Code mal formaté	Lancer `terraform fmt -recursive` localement

■ ****Fin du Lab 15 — Vous maîtrisez les bonnes pratiques avancées et le CI/CD Terraform !****

*Félicitations — vous avez complété les **15 labs** de la formation Terraform ! ■*